PROJECT REPORT

FreshMart Retail Insights — Stock Health Report

SQL Data Analytics Assignment

Author	Manikanta Kumar Golla
Email	manikantakumargolla4@gmail.com
Subject	Database Management / SQL
Date	April 2026

1. Executive Summary

FreshMart is a regional supermarket chain facing significant financial losses caused by inventory mismanagement — specifically overstocking perishables that expire unsold, and accumulating dead stock that ties up working capital without generating revenue.

This project delivers a fully functional MySQL-based Stock Health Reporting System designed to address these challenges through three targeted analytical reports:

- Expiring Soon Alert — surfaces high-stock perishables expiring within 7 days
- Dead Stock Analysis — identifies SKUs with zero sales activity over 60 days
- Revenue Contribution Report — ranks product categories by last-month earnings

The solution was built using a normalized three-table relational schema, populated with realistic dummy data across 8 product categories and 29 SKUs, and validated using standard MySQL 8+ syntax including window functions.

2. Problem Statement

The Inventory Manager at FreshMart requires a data-driven 'Stock Health Report' to answer three critical business questions:

2.1 Expiring Inventory Risk

Perishable products with large stock quantities approaching their expiry date represent direct financial losses. Any item with StockCount > 50 expiring within the next 7 days needs immediate action — markdowns, promotions, or redistribution.

2.2 Dead Stock Accumulation

Products that have not recorded a single sale in the past 60 days are considered dead stock. They occupy shelf and warehouse space, freeze working capital, and may indicate discontinued or poorly placed products.

2.3 Category Revenue Visibility

Without knowing which product category drives the most revenue each month, procurement and promotional budgets cannot be allocated efficiently. A monthly category-level revenue ranking is essential for strategic decision-making.

3. Database Schema Design

The schema follows Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. Three tables were designed:

3.1 Categories Table

Column	Data Type	Constraint	Description
CategoryID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique category identifier
CategoryName	VARCHAR(80)	NOT NULL, UNIQUE	Name of the product family
Department	VARCHAR(60)	NOT NULL	Store department grouping

3.2 Products Table

Column	Data Type	Constraint	Description
ProductID	INT	PRIMARY KEY	Unique SKU identifier
ProductName	VARCHAR(120)	NOT NULL	Full product display name
CategoryID	INT	FOREIGN KEY	Links to Categories table
UnitPrice	DECIMAL(8,2)	CHECK > 0	Retail shelf price
StockCount	INT	CHECK >= 0	Current units on shelf
ExpiryDate	DATE	NULL allowed	NULL for non-perishables
AddedOn	DATE	DEFAULT CURDATE()	Date SKU was catalogued

3.3 SalesTransactions Table

Column	Data Type	Constraint	Description
TransactionID	INT	PRIMARY KEY	Auto-generated line-item ID
ProductID	INT	FOREIGN KEY	Links to Products table
QuantitySold	INT	CHECK > 0	Units sold in transaction
SalePrice	DECIMAL(8,2)	CHECK > 0	Actual price charged
SaleDate	DATE	DEFAULT CURDATE()	Date of the sale
StoreLocation	VARCHAR(80)	NULL allowed	Branch where sale occurred

3.4 Entity-Relationship Overview

```
Categories (1) ──< Products (1) ──< SalesTransactions
```

One Category can contain many Products. One Product can appear in many SalesTransactions. Both relationships are enforced with InnoDB FOREIGN KEY constraints, ensuring no orphaned records exist in the database.

4. Report Query Design

4.1 Report 1 — Expiring Soon Alert

Business Goal: Flag products that will expire within 7 days and still hold more than 50 units, giving staff time to apply markdowns before the stock becomes a write-off.

SQL Logic:

- WHERE ExpiryDate BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 7 DAY)
- AND StockCount > 50
- Computed column: DATEDIFF(ExpiryDate, CURDATE()) → DaysUntilExpiry
- Computed column: StockCount × UnitPrice → StockAtRiskValue (monetary exposure)
- Ordered by urgency: soonest expiry first, highest value second

Sample Output Columns: ProductName | CategoryName | StockCount | ExpiryDate | DaysUntilExpiry | StockAtRiskValue

4.2 Report 2 — Dead Stock Analysis

Business Goal: Surface products that have generated zero transactions in the past 60 days so the purchasing team can halt re-orders and plan clearance.

SQL Logic:

- Subquery isolates all ProductIDs with at least one sale in the 60-day window
- NOT IN operator excludes active products from the result set
- AND StockCount > 0 filters out already-depleted items
- TiedUpCapital = StockCount × UnitPrice quantifies the financial impact
- Ordered by capital tied up (descending) to prioritise clearing highest-value dead stock first

Note: Products with only older transactions (> 60 days ago) are correctly classified as dead stock, validating boundary condition handling.

4.3 Report 3 — Revenue Contribution by Category

Business Goal: Rank all product categories by gross revenue generated in the previous complete calendar month to inform promotional and procurement priorities.

SQL Logic:

- SUM(QuantitySold × SalePrice) GROUP BY CategoryID calculates gross revenue
- DATE_FORMAT(..., '%Y-%m-01') anchors the filter to the first day of the prior month
- Window function SUM(...) OVER () computes the grand total for share percentage
- RevenueSharePct = CategoryRevenue / GrandTotal × 100 reveals relative contribution
- COUNT(DISTINCT TransactionID) shows transaction volume alongside revenue

5. Dummy Data Overview

Realistic test data was designed to exercise all three report queries meaningfully:

Dataset	Volume	Purpose
Categories	8 records	Cover perishable, dry, frozen & non-food
Products	29 SKUs	Mix of perishables and shelf-stable items
Active Transactions	29 rows	Sales within last 30 days — active sellers
Aged Transactions	3 rows	Sales > 60 days ago — boundary test data
Dead Stock Items	11 products	Zero recent sales — appear in Report 2
Expiring Items	~9 products	Expiry within 7 days — appear in Report 1

Key design decision: Three products (Organic Butter, Croissants, Mixed Nuts) carry intentional aged transactions (65–80 days old) to verify that the Dead Stock 60-day boundary filter correctly classifies them as inactive despite having a historical sales record.

6. Expected Report Outputs

6.1 Expiring Soon — Expected Highlights

Product	Category	Stock	Expiry	Risk Value
White Sandwich Bread	Bakery	200	Today +1	£300.00
Free-Range Eggs (12pk)	Dairy & Eggs	95	Today +2	£304.00
Full-Cream Milk 1L	Dairy & Eggs	120	Today +3	£222.00
Roma Tomatoes 500g	Fresh Produce	90	Today +3	£144.00
Greek Yoghurt 500g	Dairy & Eggs	85	Today +5	£204.00
Multigrain Loaf	Bakery	55	Today +4	£115.50

6.2 Dead Stock — Expected Highlights

Product	Category	Stock	Unit Price	Capital Tied
Gummy Bears 200g	Snacks	110	£1.90	£209.00
Toothpaste 100g	Personal Care	90	£1.95	£175.50
Dish-Washing Liquid	Household	70	£2.30	£161.00
Baby Spinach 200g	Fresh Produce	65	£2.20	£143.00
Beef Burger Patties	Frozen Foods	45	£5.50	£247.50

6.3 Revenue Contribution — Expected Category Ranking

Category	Department	Transactions	Units Sold	Est. Revenue
Bakery	Perishables	5	175	£284.50
Dairy & Eggs	Perishables	6	91	£235.05
Beverages	Dry Goods	4	95	£181.85
Fresh Produce	Perishables	4	117	£157.03
Snacks	Dry Goods	2	55	£99.00

7. Technical Specifications

Parameter	Detail
Database Engine	MySQL 8.0+ (InnoDB storage engine)

Parameter	Detail
SQL Standard	ANSI SQL with MySQL extensions
Window Functions	SUM() OVER () — requires MySQL 8.0+
Date Functions Used	CURDATE(), DATE_ADD(), DATE_SUB(), DATEDIFF(), DATE_FORMAT()
Referential Integrity	FOREIGN KEY constraints with InnoDB
Data Validation	CHECK constraints on price and quantity columns
Character Encoding	utf8mb4 / utf8mb4_unicode_ci
Script Type	Self-contained DDL + DML + SELECT
Execution Command	mysql -u root -p < freshmart_stock_health_report_mysql.sql

8. Conclusion

This project successfully delivers a complete, self-contained MySQL solution for FreshMart's inventory intelligence requirements. The three analytical reports — Expiring Soon, Dead Stock, and Revenue Contribution — directly address the business pain points of overstock waste and dead inventory accumulation.

Key technical achievements of this project:

- A clean 3NF-compliant relational schema with enforced referential integrity
- Dynamic, date-relative queries that remain accurate on any execution date
- Boundary condition handling in the Dead Stock query (60-day window)
- Monetary impact quantification (StockAtRiskValue, TiedUpCapital) to prioritise actions
- Window function usage for percentage share calculation in the revenue report
- Realistic dummy data designed to produce meaningful, non-trivial report outputs

The SQL script is production-ready and can be extended to include store-level filtering, automated alerting via stored procedures, or integration with a business intelligence dashboard such as Metabase or Tableau.

Submitted by: Manikanta Kumar Golla | manikantakumargolla4@gmail.com